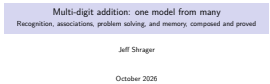
Multi-digit addition: one model from many

Recognition, associations, problem solving, and memory, composed and proved

Jeff Shrager

October 2026

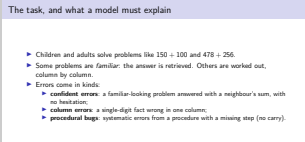
Multi-digit addition: one model from many



This talk is about the largest model in CogLean: three-digit addition, built by composing theories that were each formalized on their own. EPAM's discrimination nets recognize the problem. Siegler and Shrager's distributions of associations retrieve answers and single-digit facts. A problem solver, in the GPS and repair-theory tradition, works the columns. And the memory itself is a parameter, so ACT-R's declarative memory can be swapped in. I'll describe each component, how they compose, and what is proved about the whole that none of the parts says alone.

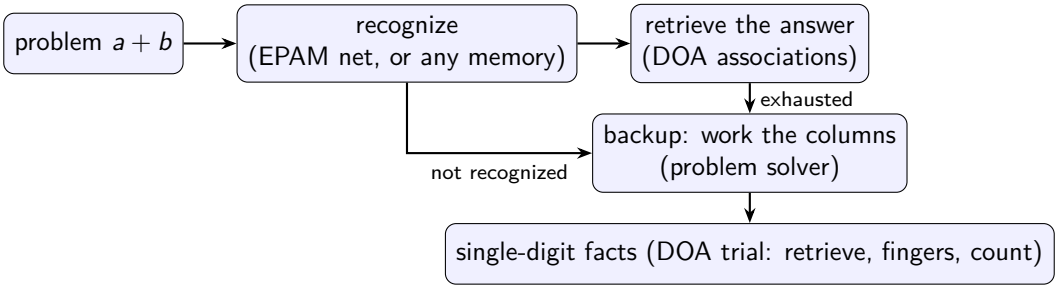
The task, and what a model must explain

- Children and adults solve problems like $150 + 100$ and $478 + 256$.
- Some problems are *familiar*: the answer is retrieved. Others are worked out, column by column.
- Errors come in kinds:
  - **confident errors**: a familiar-looking problem answered with a neighbour's sum, with no hesitation;
  - **column errors**: a single-digit fact wrong in one column;
  - **procedural bugs**: systematic errors from a procedure with a missing step (no carry).



The task is three-digit addition. Some problems are familiar and their answers are simply retrieved; others are worked out column by column. And the errors come in kinds that need different explanations. Confident errors: a problem that looks like a familiar one gets that problem's answer, with no hesitation. Column errors: a single-digit fact goes wrong in one column. And procedural bugs: systematic errors from a procedure missing a step, like never carrying. One model should produce all three, each for the right reason.

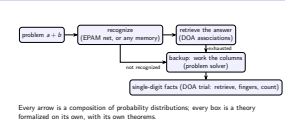
The architecture



Every arrow is a composition of probability distributions; every box is a theory formalized on its own, with its own theorems.

Multi-digit addition: one model from many

The architecture



Here's the architecture. A problem is first recognized, by an EPAM discrimination net or, since memory is a parameter, by any memory. If it's recognized, the answer is retrieved from that problem's distribution of associations. If retrieval is exhausted, or nothing is recognized, the backup takes over and works the columns, and that backup is now a problem solver. Each column needs a single-digit fact, which comes from a full distributions-of-associations trial: retrieve, or put up fingers, or count. Every arrow is a composition of probability distributions, and every box is a theory that was formalized and proved on its own first.

Component 1: recognition (EPAM)

- A **discrimination net** tests features of the problem's code, and sorts it to a stored image.
- Learning adds a test only where two codes are confused.
- A net that has never needed to tell two problems apart can't: $150 + 100$ is sorted to $100 + 100$ when the net tests only the digit where the familiar problems differ.

Proved for every net: recognition depends only on the features tested; a confusion agrees on every tested feature; learning a problem never disturbs another.

Multi-digit addition: one model from many

Component 1: recognition (EPAM)

Component 1: recognition (EPAM)

- A **discrimination net** tests features of the problem's code, and sorts it to a stored image.
- Learning adds a test only where two codes are confused.
- A net that has never needed to tell two problems apart can't: 150 + 100 is sorted to 100 + 100 when the net tests only the digit where the familiar problems differ.

Proved for every net: recognition depends only on the features tested; a confusion agrees on every tested feature; learning a problem never disturbs another.

The first component is EPAM's discrimination net. It tests features of the problem's code and sorts the problem to a stored image. Learning adds a test only where two codes are confused, so a net that has never needed to tell two problems apart can't. In our worked case, the net learned one hundred plus one hundred and one hundred plus two hundred, so it tests only the digit where they differ, and one hundred fifty plus one hundred is sorted to one hundred plus one hundred. The general theorems hold for every net: recognition depends only on tested features, a confusion agrees on all of them, and learning never disturbs other problems.

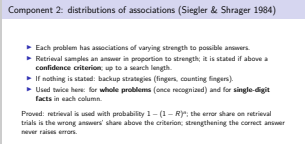
Component 2: distributions of associations (Siegler & Shrager 1984)

- Each problem has associations of varying strength to possible answers.
- Retrieval samples an answer in proportion to strength; it is stated if above a **confidence criterion**; up to a search length.
- If nothing is stated: backup strategies (fingers, counting fingers).
- Used twice here: for **whole problems** (once recognized) and for **single-digit facts** in each column.

Proved: retrieval is used with probability $1 - (1 - R)^n$; the error share on retrieval trials is the wrong answers' share above the criterion; strengthening the correct answer never raises errors.

Multi-digit addition: one model from many

Component 2: distributions of associations (Siegler & Shrager 1984)



The second component is the distributions-of-associations model that Bob Siegler and I proposed in 1984. Each problem has associations of different strengths to possible answers. Retrieval samples an answer in proportion to strength, states it if it's above a confidence criterion, and tries up to a search length; if nothing is stated, the child falls back on fingers or counting. In the combined model it's used twice: for whole problems once they're recognized, and for the single-digit fact in each column. Its own theorems hold throughout, for instance that strengthening the correct answer never raises the error rate.

Component 3: the column backup as problem solving

- A problem space: add a column (its fact from a DOA trial), write, write and carry, finish.
- **The bridge**: the solver answers exactly as the old fixed column procedure did, for every problem and every fact process.
- **Means-ends analysis** (GPS) works units first, though its goal names no order.
- **Repair theory** (Brown & VanLehn 1980): delete the carry rule, and a column sum of 10 or more is an impasse; its repairs are bugs (no carry: $15 + 17 = 22$).

Component 3: the column backup as problem solving

- A problem space: add a column (its fact from a DOA trial), write, write and carry, finish.
- **The bridge**: the solver answers exactly as the old fixed column procedure did, for every problem and every fact process.
- **Means-ends analysis** (GPS) works units first, though its goal names no order.
- **Repair theory** (Brown & VanLehn 1980): delete the carry rule, and a column sum of 10 or more is an impasse; its repairs are bugs (no carry: $15 + 17 = 22$).

The third component is new: the column backup as problem solving. Its operators are adding a column, with the fact from a distributions-of-associations trial, writing, writing and carrying, and finishing. A bridge theorem shows it answers exactly as the old fixed procedure did, for every problem, so everything proved before carries over. Means-ends analysis discovers the units-first order on its own. And with the carry rule deleted, as repair theory does, a column sum of ten or more is an impasse, and its repairs are bugs: fifteen plus seventeen becomes twenty-two.

Component 4: memory as a parameter

- Recognition is any **retriever**: study, then retrieve a distribution over stored items.
- EPAM's net is one; ACT-R's declarative memory (activation, partial matching) is another.
- Three **laws** of memory, determinacy, certainty and stability, are enough for confident errors; EPAM's net has all three, ACT-R has none.

So which errors a learner makes depends on which memory they have, and the laws say why.

Component 4: memory as a parameter

- Recognition is any **retriever**: study, then retrieve a distribution over stored items.
- EPAM's net is one; ACT-R's declarative memory (activation, partial matching) is another.
- Three **laws** of memory, determinacy, certainty and stability, are enough for confident errors; EPAM's net has all three, ACT-R has none.

So which errors a learner makes depends on which memory they have, and the laws say why.

The fourth component is that memory is a parameter. Recognition is stated for any retriever, anything you can study items into and retrieve from. EPAM's net is one, and ACT-R's declarative memory, with activation and partial matching, is another. Three laws of memory, determinacy, certainty and stability, are enough to guarantee confident errors. EPAM's net has all three; ACT-R has none of them. So the kind of errors a learner makes depends on the kind of memory they have, and the laws say exactly why.

How they compose

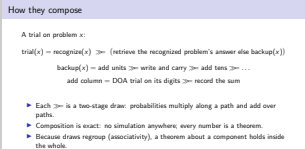
A trial on problem x:

$$\text{trial}(x) = \text{recognize}(x) \gg\!= (\text{retrieve the recognized problem's answer else backup}(x))$$

$$\text{backup}(x) = \text{add units} \gg\!= \text{write and carry} \gg\!= \text{add tens} \gg\!= \ldots$$

$$\text{add column} = \text{DOA trial on its digits} \gg\!= \text{record the sum}$$

- Each $\gg\!=$ is a two-stage draw: probabilities multiply along a path and add over paths.
- Composition is exact: no simulation anywhere; every number is a theorem.
- Because draws regroup (associativity), a theorem about a component holds inside the whole.



How do they compose? A trial is recognition, then either retrieval of the recognized problem's answer or the backup. The backup is a chain of problem-solving steps, and adding a column is itself a full distributions-of-associations trial on that column's digits. Each step is a two-stage draw: probabilities multiply along a path and add over paths. It's exact, with no simulation anywhere, and every number is a theorem. And because these draws regroup freely, a theorem proved about a component holds where that component sits inside the whole.

An application built from the library

Layer	What it supplies	Where
Library	discrimination nets and their learning; sampled retrieval with a criterion; recognize, then retrieve, else back up; the decision cycle with impasses and repairs; exact distributions	`CogLib/`
Theory	the single-digit trial: retrieve, fingers, count (Siegler & Shrager)	`StrategyChoice/`
Task knowledge	the problem's code; the column problem space and its procedure	`MultiDigit/`
The application	the wiring: the solver as the backup, a DOA trial as each column's fact	`Integrated.lean`

- No new mechanism, and no answer given: correct knowledge enters only as single-digit associations and the carry rule.
- It inherits every theorem of its parts, and proves its own about the whole.

Multi-digit addition: one model from many

An application built from the library

An application built from the library

Layer	What it supplies	Where
Library	discrimination nets and their learning; sampled retrieval with a criterion; recognize, then retrieve, else back up; the decision cycle with impasses and repairs; exact distributions	`CogLib/`
Theory	the single-digit trial: retrieve, fingers, count (Siegler & Shrager)	`StrategyChoice/`
Task knowledge	the problem's code; the column problem space and its procedure	`MultiDigit/`
The application	the wiring: the solver as the backup, a DOA trial as each column's fact	`Integrated.lean`

- No new mechanism, and no answer given: correct knowledge enters only as single-digit associations and the carry rule.
- It inherits every theorem of its parts, and proves its own about the whole.

This is the point I most want to make. The integrated model is an application: something built out of the library, not a new model written from scratch. The library supplies the mechanisms: discrimination nets and how they learn, sampled retrieval with a confidence criterion, the general pattern of recognize, then retrieve, else back up, the decision cycle with impasses and repairs, and exact probability distributions. A theory package supplies the single-digit trial from Siegler and Shrager. The task supplies its own knowledge: how a problem is coded, and the column problem space with its procedure. And the application itself is just the wiring: the problem solver goes in the backup slot, and a single-digit trial supplies each column's fact. There's no new mechanism, and nothing is handed the answer; correct knowledge enters only as single-digit associations and the carry rule. And the application inherits every theorem of its parts, and then proves its own theorems about the whole.

What rests on what

Kind	Results
Follows from the composition, at every setting	the solver answers as the column backup; whole-problem errors are untouched by the solver, so a better solver cannot repair them; a solver error is a single-digit error moved to its column
Given stated conditions	confident errors, and two sources of error on a confused problem, given: the confusion (derived from the learned net), sharp associations (assumed, not yet explained), positive strengths (well-formedness)
Illustrative numbers	the worked case, next

No phenomenon is a premise of any of these theorems.

Multi-digit addition: one model from many

What rests on what

What rests on what

Kind	Results
Follows from the composition, at every setting	the solver answers as the column backup; whole-problem errors are untouched by the solver, so a better solver cannot repair them; a solver error is a single-digit error moved to its column
Given stated conditions	confident errors, and two sources of error on a confused problem, given: the confusion (derived from the learned net), sharp associations (assumed, not yet explained), positive strengths (well-formedness)
Illustrative numbers	the worked case, next

No phenomenon is a premise of any of these theorems.

Here is what each result rests on, because that's where the explanatory work is. Some results follow from the composition alone, at every setting: the solver answers exactly as the old column backup did; whole-problem errors don't depend on the solver at all, so no improvement to the solver can repair a confident error; and a solver error is a single-digit error moved to its column. Others need stated conditions. Confident errors, and the fact that a confused problem has two sources of error with different signatures, need the confusion, which here is derived from the net EPAM learned; sharp associations, which are still an assumption we haven't explained; and positive strengths, which is just well-formedness. And then there are the numbers in the worked case, which are only an illustration. In none of these theorems is the phenomenon itself a premise.

What the whole predicts

Prediction	From
Confident errors: every retrieval of a confused problem is wrong, with no extra backup	recognition (EPAM laws) + sharp associations (DOA)
A column error is a single-digit error moved to its place	problem solver + DOA facts
Carrying adds no error of its own	problem solver (bridge)
Procedural bugs only on carry problems	problem solver without the carry rule
Learning removes a confusion without disturbing other problems	EPAM learning

Multi-digit addition: one model from many

What the whole predicts

What the whole predicts

Prediction	From
Confident errors: every retrieval of a confused problem is wrong, with no extra backup	recognition (EPAM laws) + sharp associations (DOA)
A column error is a single-digit error moved to its place	problem solver + DOA facts
Carrying adds no error of its own	problem solver (bridge)
Procedural bugs only on carry problems	problem solver without the carry rule
Learning removes a confusion without disturbing other problems	EPAM learning

Here's what the whole model predicts, and which parts each prediction comes from. Confident errors come from recognition, whose laws make the confusion certain, together with sharp associations. A column error is a single-digit error moved to its place, from the problem solver using distributions-of-associations facts. Carrying adds no error of its own, through the bridge. Procedural bugs appear only on carry problems, from the problem solver without its carry rule. And learning removes a confusion without disturbing other problems, from EPAM. Each row needs at least two theories; none is in any single one.

A worked case

	EPAM	ACT-R
$150 + 100$ answered 200 (neighbour's sum) by retrieval	45/46	225/529
$150 + 100$ reaches the solver	1/46	383/1058
$150 + 100$ answered correctly, by the solver	$\approx .014$	$\approx .238$
$15 + 17$, no carry rule, strict critics	22, for every such learner	

- Values are exact, at settings chosen for illustration; the predictions above hold for every setting. Each column's facts follow one chosen rule.
- The solver alone on $150 + 100$: 250 $\approx .66$; 260, 350, 251 $\approx .10$ each, single-digit errors in place (tens, hundreds, units).
- With EPAM, the confusion is certain; with ACT-R, it is graded.

A worked case

	EPAM	ACT-R
$150 + 100$ answered 200 (neighbour's sum) by retrieval	45/46	225/529
$150 + 100$ reaches the solver	1/46	383/1058
$150 + 100$ answered correctly, by the solver	$\approx .014$	$\approx .238$
$15 + 17$, no carry rule, strict critics	22, for every such learner	

- Values are exact, at settings chosen for illustration; the predictions above hold for every setting. Each column's facts follow one chosen rule.
- The solver alone on $150 + 100$: $250 \approx .66$; 260, 350, $251 \approx .10$ each, single-digit errors in place (tens, hundreds, units).
- With EPAM, the confusion is certain; with ACT-R, it is graded.

A worked case, with every answer computed. With EPAM recognition and illustrative strengths, one hundred fifty plus one hundred is answered with its neighbour's sum, two hundred, by retrieval forty-five times in forty-six. Only once in forty-six does it reach the problem solver, and the solver, adding each column with a single-digit trial, gets two hundred fifty about two thirds of the time. So it's answered correctly only about one and a half percent of the time. The solver's own errors are single-digit errors in place: two sixty, three fifty, two fifty-one, each about ten percent. Swap in ACT-R's memory, with nothing else changed, and the confusion is graded: the solver is reached more often, and the problem is answered correctly about a quarter of the time. And with the problem solver missing its carry rule and strict critics, fifteen plus seventeen is twenty-two. These numbers are exact but come from chosen settings, including one rule for every column's single-digit associations; the predictions on the earlier slides hold for every setting.

Is the formalization faithful?

- A prose specification, with no Lean in it, and preregistered predictions.
- A fresh AI agent, given only the prose, wrote a Python simulator.
- **No failures**: 243 outcome frequencies on 4 instances matched Lean's exact values; every applicable prediction passed.
- It also found five vacuous predictions and a gap in the formalization, since fixed or recorded.

The blind check predates the problem solver; by the bridge, its results carry over unchanged.

Is the formalization faithful?

- A prose specification, with no Lean in it, and preregistered predictions.
- A fresh AI agent, given only the prose, wrote a Python simulator.
- **No failures**: 243 outcome frequencies on 4 instances matched Lean's exact values; every applicable prediction passed.
- It also found five vacuous predictions and a gap in the formalization, since fixed or recorded.

The blind check predates the problem solver; by the bridge, its results carry over unchanged.

Is the Lean model the model described in prose? We checked with a blind simulation. A prose specification with no Lean in it, predictions preregistered before any simulator existed, and a fresh AI agent that wrote a Python simulator from the prose alone. There were no failures: two hundred forty-three outcome frequencies matched Lean's exact values. It also found five vacuous predictions and a gap in the formalization. The check was run before the problem solver existed, but because of the bridge theorem, its results carry over unchanged.

What each component contributes

Component	Source	Role in the whole
Discrimination nets	EPAM (Feigenbaum; Simon)	which problems look alike; certain confusions
Distributions of associations	Siegler & Shrager 1984	answers, confidence, backup choice; single-digit facts
Problem solving	GPS; Brown & VanLehn 1980	the column procedure; its order; its bugs
Retrievers and laws	ACT-R (Anderson); laws new here (the theory matrix)	which memory gives which errors, and why

Each was formalized and proved on its own; the composition is proved too.

What each component contributes

Component	Source	Role in the whole
Discrimination nets	EPAM (Feigenbaum; Simon)	which problems look alike; certain confusions
Distributions of associations	Siegler & Shrager 1984	answers, confidence, backup choice; single-digit facts
Problem solving	GPS; Brown & VanLehn 1980	the column procedure; its order; its bugs
Retrievers and laws	ACT-R (Anderson); laws new here (the theory matrix)	which memory gives which errors, and why

Each was formalized and proved on its own; the composition is proved too.

To summarize the parts. Discrimination nets, from EPAM, say which problems look alike and make confusions certain. Distributions of associations, from Siegler and Shrager, give answers, confidence and the choice of backup, and also the single-digit facts. Problem solving, from GPS and repair theory, gives the column procedure, its order and its bugs. And retrievers with laws of memory say which memory gives which errors, and why. Each was formalized and proved on its own, and the composition is proved too.

Limits, and next

- Worked cases use chosen settings; priors, whole-problem and single-digit, should come from experience, as Siegler & Shrager's did from counting (open issue).
- "Multi-digit" is three-digit.
- **Chunking** would close the loop: a fact found by counting at a retrieval impasse becomes an association, DOA's learning rule as problem-solving learning.
- Subtraction, borrowing, and repair theory's own bug data.

Limits, and next

- Worked cases use chosen settings; priors, whole-problem and single-digit, should come from experience, as Siegler & Shrager's did from counting (open issue).
- "Multi-digit" is three-digit.
- **Chunking** would close the loop: a fact found by counting at a retrieval impasse becomes an association, DOA's learning rule as problem-solving learning.
- Subtraction, borrowing, and repair theory's own bug data.

Limits. The worked cases use settings chosen for illustration; the prior associations should come from experience, as Siegler and Shrager's came from counting, and that's an open issue. Multi-digit here means three-digit. Next, chunking would close the loop between the parts: a fact found by counting when retrieval reaches an impasse becomes an association, which is the distributions-of-associations learning rule seen as problem-solving learning. And subtraction, with borrowing, where repair theory's bug data are.